

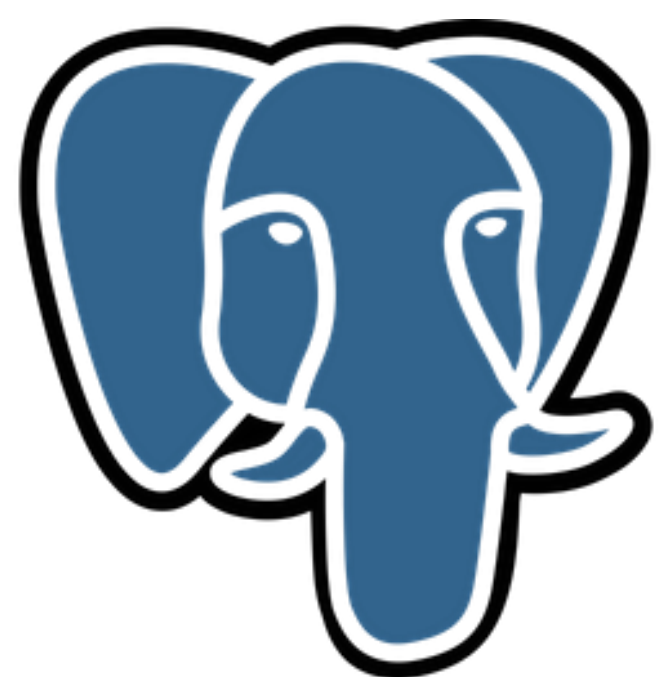
Exploring Improvements to B-Tree Indexes for Database Optimization

Uriel Benymon, Paul Blaer PhD

RISE Research Program | Department of Computer Science | Columbia University

Introduction:

Information for student records, financial services, healthcare systems, and even social media profiles are all largely managed by databases and the data structures inside them. Data structures are programs that computers use to efficiently manage and store data. They are fundamental for databases, which serve the same purpose but on larger scales. Databases use data structures called indexes, and one is called the B-Tree. Indexes perform operations including searching, inserting, and deleting data inside the columns and rows of databases. These operations can also be performed by multiple people within the same database, which is called concurrency. While indexes such as the B-Tree are efficient, concurrency puts additional pressure on them when searching, inserting, and deleting data to maintain optimal performance and database quality, which are the core topics of this research study.



Methods:

- Gathering Tools & Database Selection**
 - Chose a database called PostgreSQL for the research process
 - PostgreSQL was chosen for being open-source and offering multiple indexes
 - B-Tree, Hash, and BRIN indexes were selected to test
 - Niche indexes such as GiST and SP-GiST were omitted
- Developing the Benchmarking Environment**
 - Developed a benchmarking environment on PostgresSQL 17.5 to steer the project
 - Search, insert, and delete operations were tested for the indexes
 - Java and SQL were used to time the indexes in milliseconds and nanoseconds
 - Realistic data for people were tested using text, integer, datatypes etc.
- Benchmarking and Comparison Analyses**
 - The three images in the center are the results of the index tests (**Figures 1, 2, and 3**)
 - Hash, B-Tree, BRIN, and sequential (no index) showed varying behaviors from small to large loads
 - Insert and delete times showed increased time costs for indexes
 - Research continued with the B-Tree index as the focus due to its behaviors
- Native and Modified B-Tree & Environment**
 - Created a modified B-Tree and environment for comparison with the standard B-Tree
 - Simulated 64 clients in PostgreSQL for concurrency tests
 - The duration of the research focused on the B-Tree insertion times and analysis

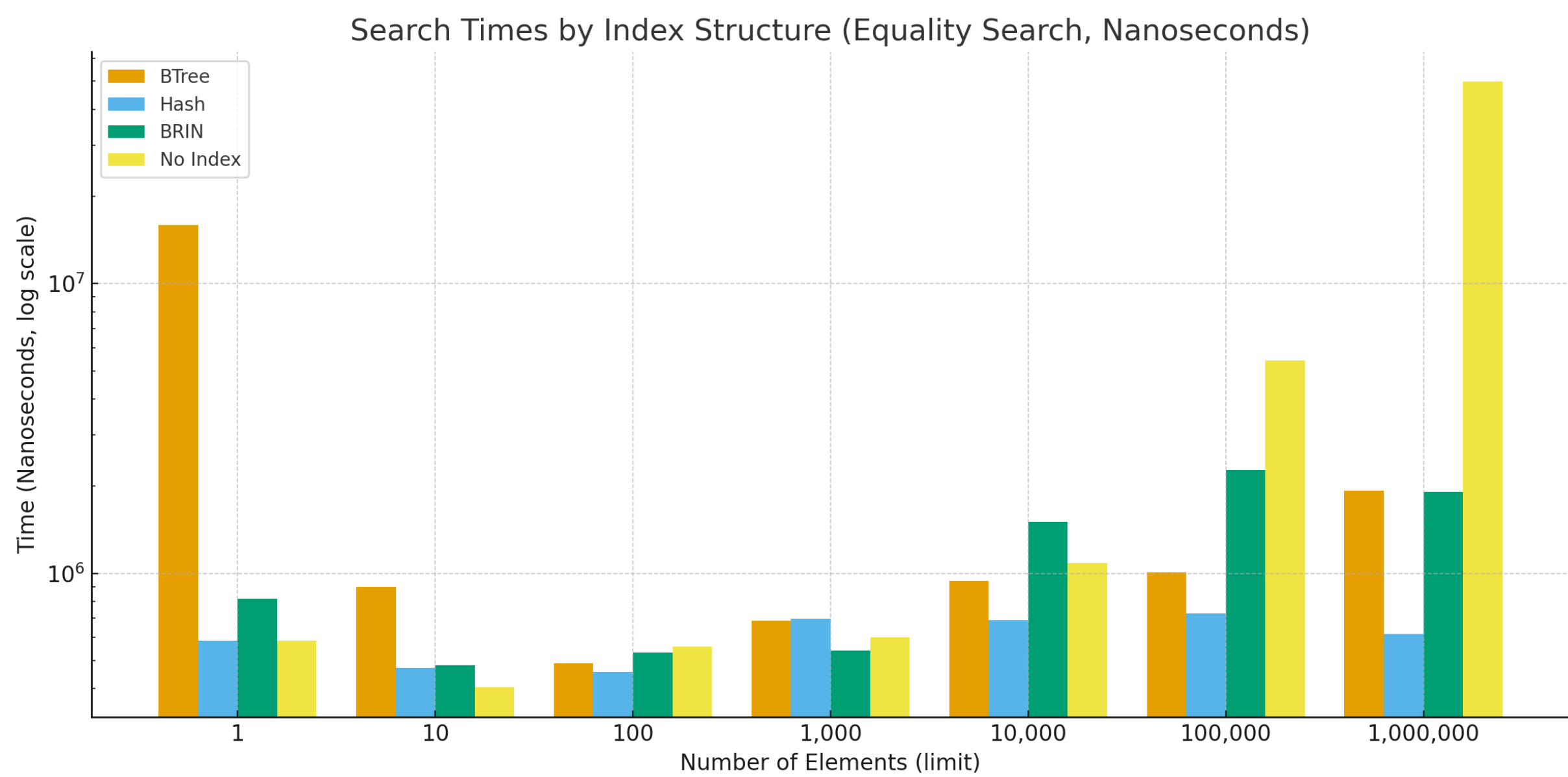


Figure 1)

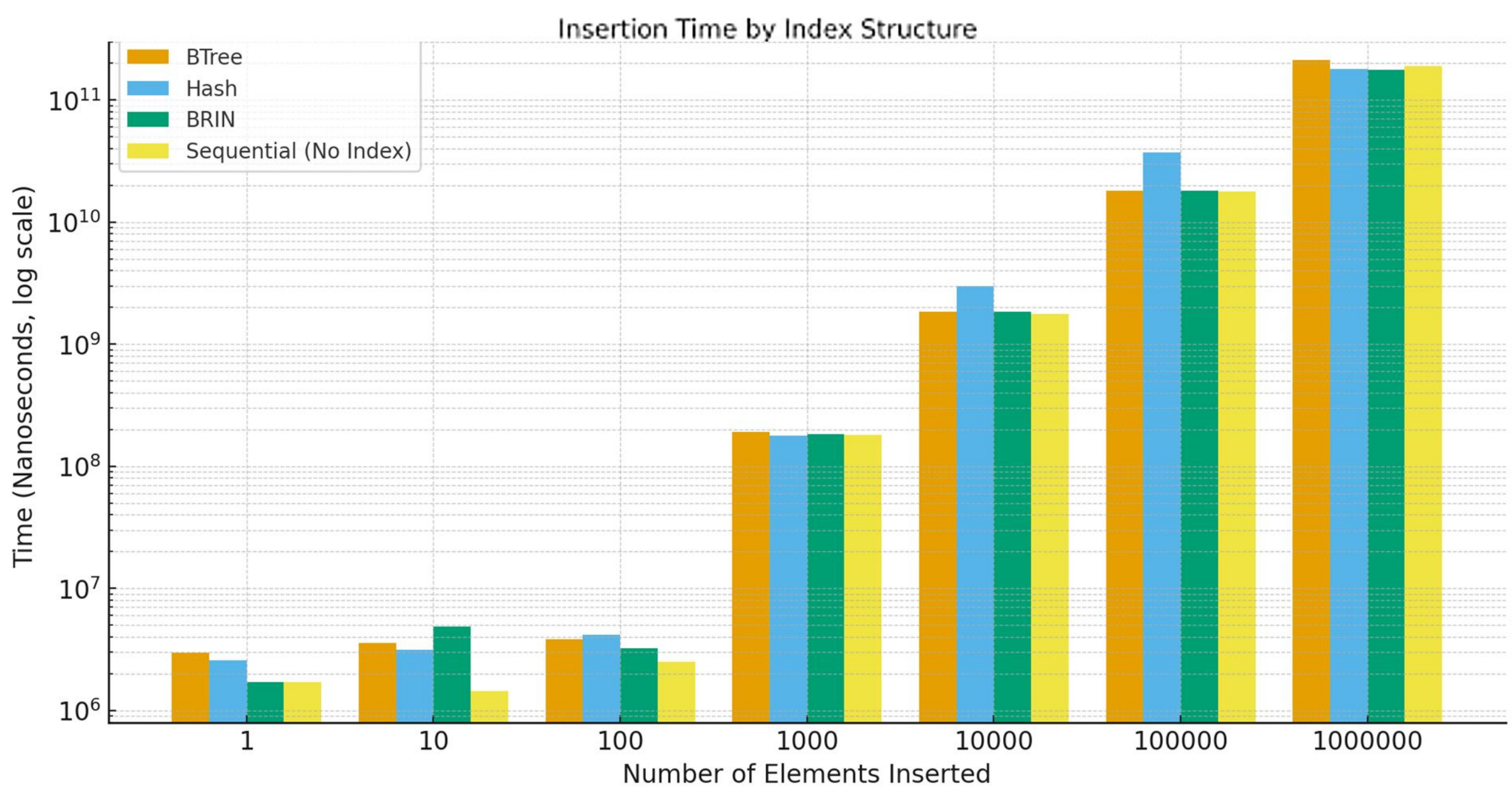


Figure 2)

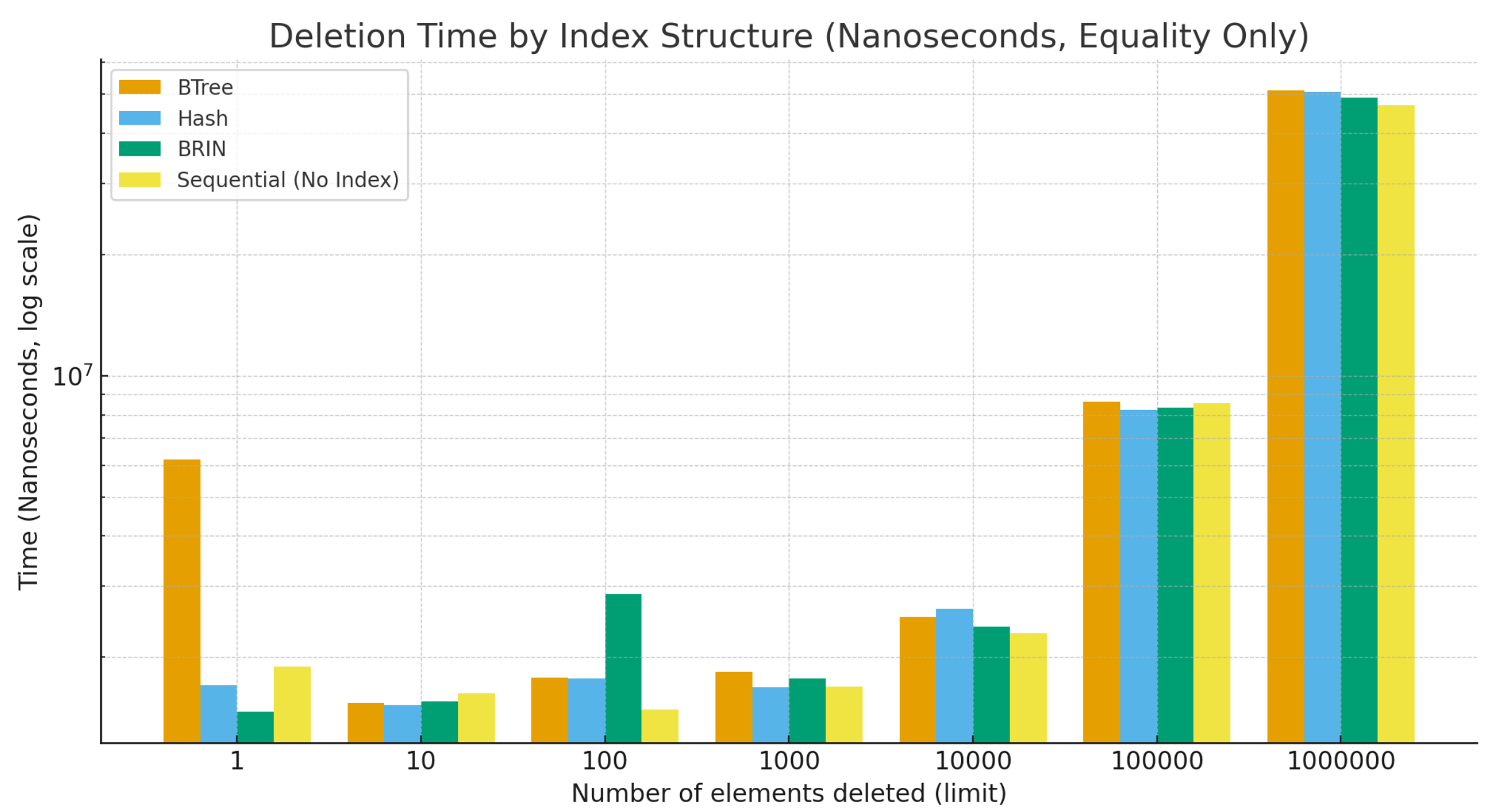


Figure 3)

Results:

- Native vs Modified B-Tree Insertion**
 - The results of the native vs modified B-Tree insertion tests are in **Figure 4)**
 - The tests showed ~30% higher transactions per second (TPS) for the modified B-Tree. TPS tracks the number of insertion writes per second
 - A ~9% lead was maintained when approaching 64 clients for the modified B-Tree
- B-Tree Trade-offs**
 - Results showed improved insertion TPS, but the tree's height increases faster with the changes made, which may affect search and deletion times
- Findings**
 - Findings suggest that augments to B-Trees' parameters, write functionality, and split conditions work under concurrent loads to hasten insertion speed, offering the possibility for phenomena this to be examined in other databases.

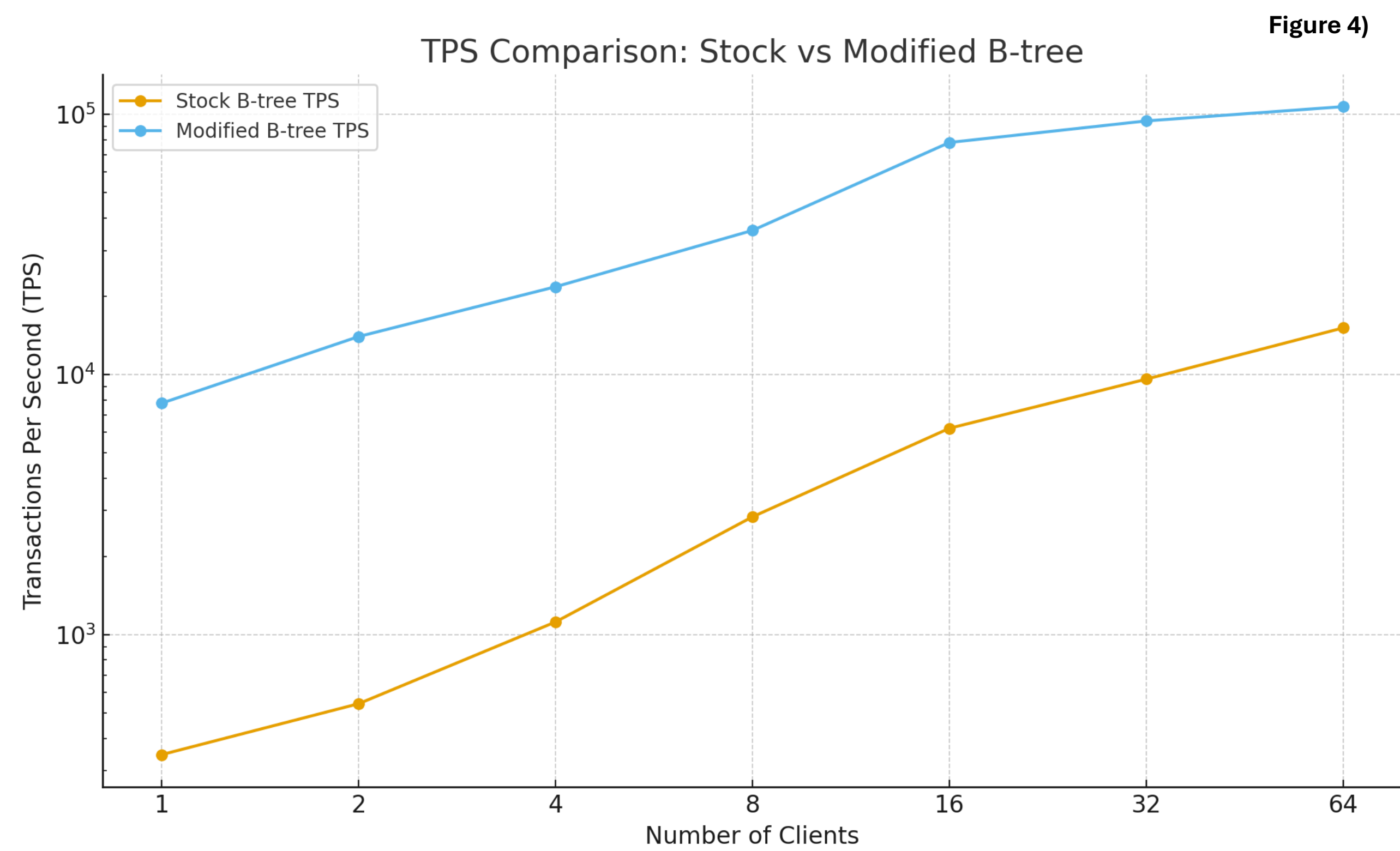


Figure 4)

Conclusions:

- Generalizations of Computer Specs**
 - The results show that individual computer specifications and B-Tree structural choices play a large role in database efficiency under concurrent loads.
- Split Optimization**
 - The B-Tree's split parameters scale optimally depending on the type of desired operations and data
- Consistent Scaling**
 - Evidence was shown in the chart above that increased node capacity and split parameter alterations scaled significantly with write speeds across clients
- Future Room for Adaptive Improvements**
 - This research opens the door to adaptive structural index changes depending on the use cases for the user(s) and the data being managed

Acknowledgements:

I would like to thank Paul Blaer for all the guidance and mentorship throughout this project. We also thank Columbia University's GS RISE Program for its grant to support in funding the project. In addition, we thank the Department of Computer Science at Columbia University and PostgreSQL for allowing us to use computing services to maintain, develop, and experiment across varying database environments.